Clase 25 — Conmutación en la red: datagramas y circuitos virtuales

Bloque: D (Red) · **Semana:** 9 · **Duración estimada:** 105 min **Correlativa:** Clase 24 (forwarding y routing) · **Sigue con:** Clase 26 (IPv4: el datagrama y las direcciones) **Material de referencia:** Tanenbaum cap. 5 (§5.1, "Las dos familias") · RFC 791, RFC 3031, RFC 3032, RFC 4364, RFC 2211, RFC 8200, RFC 8402, RFC 8986

🎯 Al terminar esta clase vas a poder

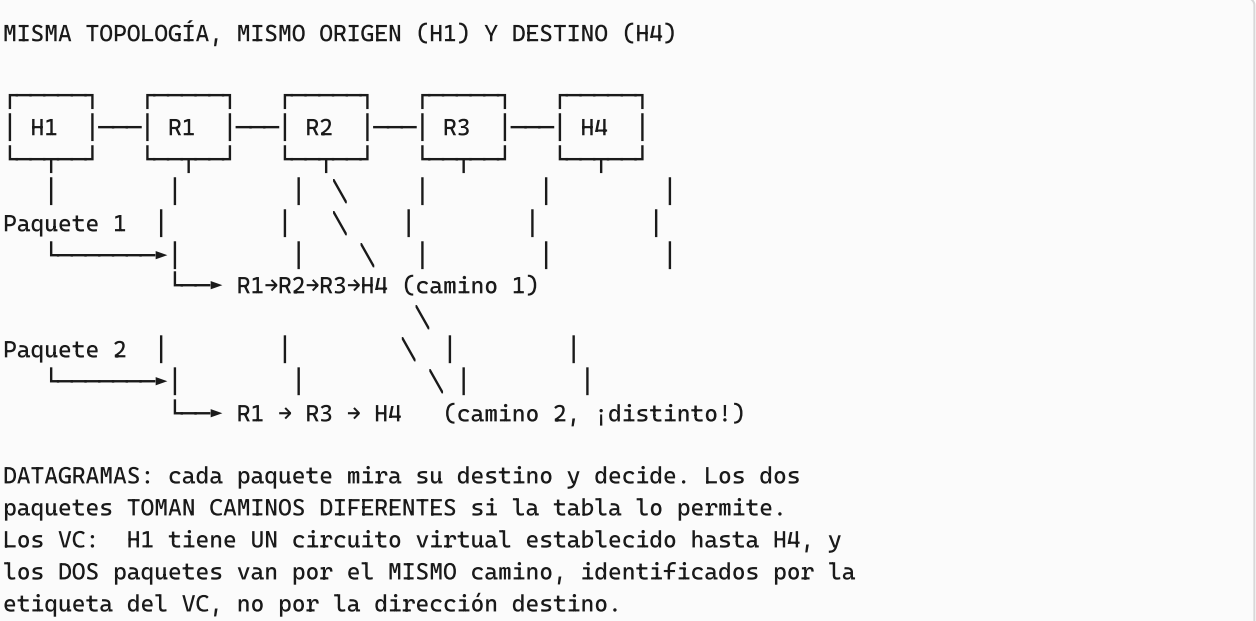
1. **Distinguir** las dos familias de conmutación de paquetes y decir **cuál de las dos usa Internet** y por qué.
2. **Describir** la conmutación de **datagramas**: sin estado, decisión por paquete, caminos distintos para paquetes del mismo flujo, y sus ventajas y desventajas.
3. **Describir** los **circuitos virtuales (VC)**: qué es la etiqueta, por qué hace falta **estado en cada router** y por qué hace falta **establecimiento previo**.
4. **Comparar** la tabla de conmutación **por VC** contra la **por prefijo**: qué se guarda en cada caso, por qué el VC es mucho más rápido de buscar y por qué **no escala**.
5. **Explicar** el establecimiento **en banda** y **fuera de banda** de un circuito virtual, con el caso de la red telefónica clásica y el del enlace dedicado.
6. **Comparar** ventajas y desventajas de datagramas y VC **desde el punto de vista de la asignación de recursos** en la red, que es exactamente lo que pregunta el examen.
7. **Calcular** cuántas entradas de tabla necesita cada esquema para una red de *n* hosts, y explicar qué se rompe cuando un enlace de un VC se cae.

> **Prerequisito del examen:** `2009-02 P4` (9 pts — datagramas vs. circuitos virtuales), `2007-12 P3` (10 pts — circuitos / paquetes / mensajes), `2010-02 P2c` (las dos formas de servicio de capa 3).

PARTE 1 — TEORÍA

1.1 Las dos familias

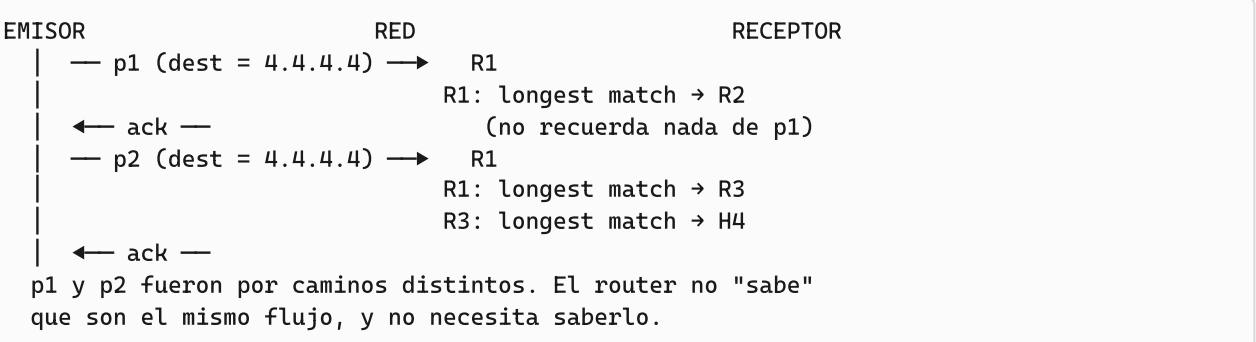
Cuando un nodo tiene que decidir **por dónde sale un paquete**, hay dos familias de diseño, y la diferencia es **una sola pregunta**: *¿la decisión depende del paquete, o depende de un camino previamente establecido?*



La tercera familia —**conmutación de mensajes**, donde el nodo reenvía la unidad completa con el nombre del destino final y el destino se reconstruye en el último salto— se estudia en la Clase 4. Lo que importa acá es que **Internet es conmutación de datagramas**, y todo lo que viene después (VC, MPLS, etiquetas) es un overlay que se apoya sobre ese datagrama.

1.2 Conmutación de datagramas

Definición operativa: cada paquete se reenvía por decisión propia, mirando su **dirección destino** y consultando la tabla (en IP, *longest prefix match*). **No hay conexión, no hay estado, no hay negociación previa.**



Ventajas.

Ventaja	Por qué
No hace falta estado	El router no memoriza nada entre paquetes. Un router que se reinicia no pierde información de la que dependan otros.
Tolera fallos	Si un enlace se cae, el siguiente paquete ya va por otro camino. La reconvergencia la maneja el plano de control, no el conmutador.
Caminos distintos por paquete	Se puede enrutar el tráfico según el estado real de la red, e incluso repartir un flujo entre enlaces (hoy es lo normal con ECMP).
Multiplexado estadístico	Los enlaces se comparten dinámicamente entre todos los flujos, y la capacidad se aprovecha mejor (§1.6 y §2.1).
Arranque instantáneo	El emisor manda el primer paquete sin handshake ni señalización. Es lo que permite que un host de la calle funcione.
Tabla chica	Una entrada por **prefijo**, y los prefijos se **agregan**: 1000 hosts en la misma /24 son **una** entrada.

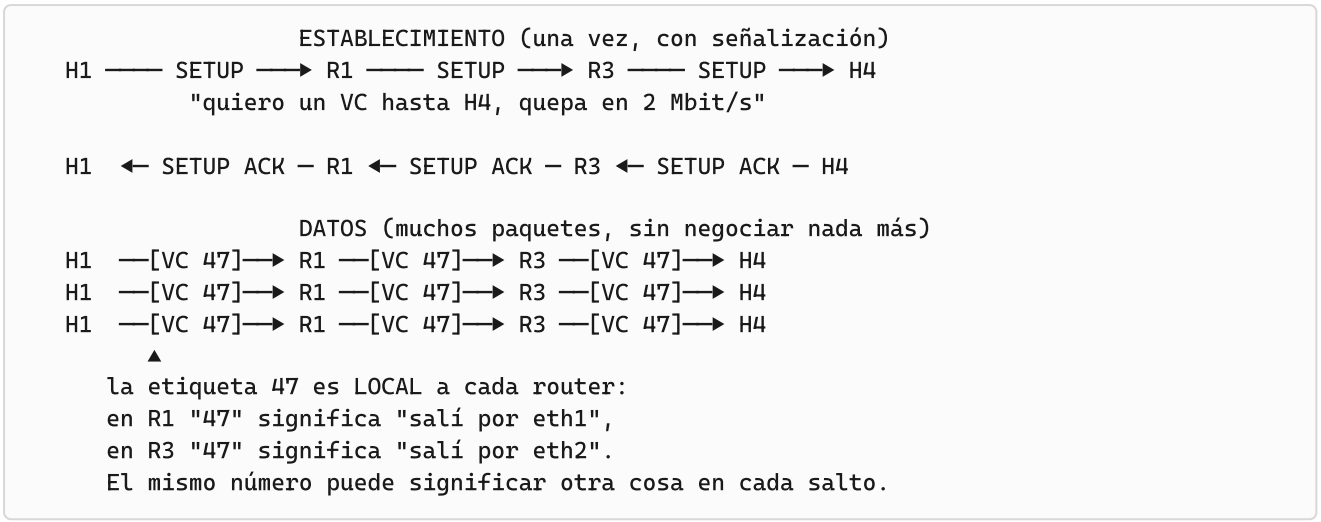
Desventajas.

Desventaja	Por qué
Decisión por paquete = trabajo por paquete	Hay que leer la cabecera, hacer el *longest prefix match* y elegir interfaz **para cada paquete**. Con VC, la búsqueda es una etiqueta exacta y más rápida.
Reordenamiento	Si dos paquetes del mismo flujo toman caminos distintos, el segundo puede llegar antes que el primero. TCP lo reordena; una aplicación de audio, no.
Cabecera grande	Cada paquete lleva la **dirección destino completa** (y en general también la de origen). Con VC sólo necesita la **etiqueta**, que es un número corto.
Sin garantía de nada	Es *best-effort*: no hay reserva de ancho de banda, de buffer ni de retardo. Un flujo puede tener la mala suerte de encontrar la red ocupada y ser descartado.
Setup con costo	En datagramas, cuando cambia un camino hay que recalcular y volver a anunciar rutas. Con VC, un cambio de camino es un evento raro y ruidoso.

Internet entero es esto. IP no establece nada: el primer paquete sale sin handshake. Cada router decide solo, por prefijo. Si dos paquetes del mismo flujo van por caminos distintos, es **el comportamiento normal, no una excepción**.

1.3 Circuitos virtuales

Definición operativa: se establece un **camino lógico** (*virtual circuit*, VC) y **todos** los paquetes de esa conexión lo siguen, identificados por una **etiqueta de VC** (*VC label*), que es **un número, no una dirección**. Cada router del camino guarda **estado** para ese circuito: a qué puerto va y qué hacer con las etiqueta que llegan por él.



Tres condiciones que un VC necesita y que el datagrama no:

1. **Estado en cada router del camino.** El router tiene que tener una entrada que diga "etiqueta 47 → puerto de salida, y esto es lo que hay que hacer con los paquetes que me lleguen con 47". Sin esa tabla no puede conmutar. Ese estado **no existe** en un router de datagramas.
2. **Establecimiento previo.** Antes del primer paquete de datos hay un **intercambio de mensajes de señalización** que recorre el camino del destino hacia el origen y vuelve. Ese intercambio, en sí, es un datagrama: la red por la que viaja el setup es una red de datagramas.
3. **Identificador en la cabecera.** El paquete lleva la etiqueta, que puede ser más corta que la dirección. Esto es lo que hace que la búsqueda en la tabla sea trivial.

Qué es la etiqueta, en concreto. En el ejemplo de arriba la etiqueta "47" **sólo significa algo dentro de cada router**. No es una dirección global, no se puede intergenerational, y el router de ingreso y el de egreso pueden coincidir o no con los extremos. Ese es el punto clave: **con VC se reemplaza el problema "¿a dónde va este paquete?" por el problema mucho más fácil "¿qué hago con esta etiqueta?"**

Dónde existen los VC. Frame Relay (**PVC**, *Permanent Virtual Circuit*, y **SVC**, *Switched Virtual Circuit*), **ATM** (con su *label switching*), **MPLS** (`RFC 3031` arquitectura, `RFC 3032` la pila de etiquetas), X.25. Y en IPv6, el **Flow Label** de la cabecera (`RFC 8200`), que es el intento estandarizado de tener "algo de VC" sin salir del datagrama: por defecto vale **0** (o sea, no se usa), y ningún router de Internet lo interpreta para reenviar.

1.4 La tabla de conmutación: por VC o por prefijo

Acá está el contraste que más se pregunta.

	Conmutación de datagramas	Conmutación por VC
Clave de la tabla	**prefijo de dirección** (con longitud)	**etiqueta de VC** (un número)
Qué se guarda como valor	enlace o puerto de salida (+ next hop, métricas, ECMP)	**número de VC de salida** en el siguiente router, y la acción a ejecutar (reenviar / encapsular / entregar)
Tipo de búsqueda	**longest prefix match**: hay que comparar contra **todas** las entradas y quedarse con la más específica	**índice directo**: `tabla[etiqueta]`, una **lectura de memoria**
Coste de la búsqueda	O(n) comparaciones, en TCAM, en paralelo	**O(1)**, una lectura
Tamaño de la clave	hasta **32 bits + longitud**	**4 bits** (MPLS: 20 bits)
Cantidad de entradas	**una por prefijo agregado** (poco, y crece con los prefijos de Internet)	**una por circuito vigente** (mucho: es el número de conexiones simultáneas)
Se puede cambiar en caliente	no: cambiar la ruta es cambiar la tabla de todos	no: hay que renegociar el circuito
Tolera que un router se reinicie	sí, no tiene estado	no: hay que **re-establecer** los circuitos

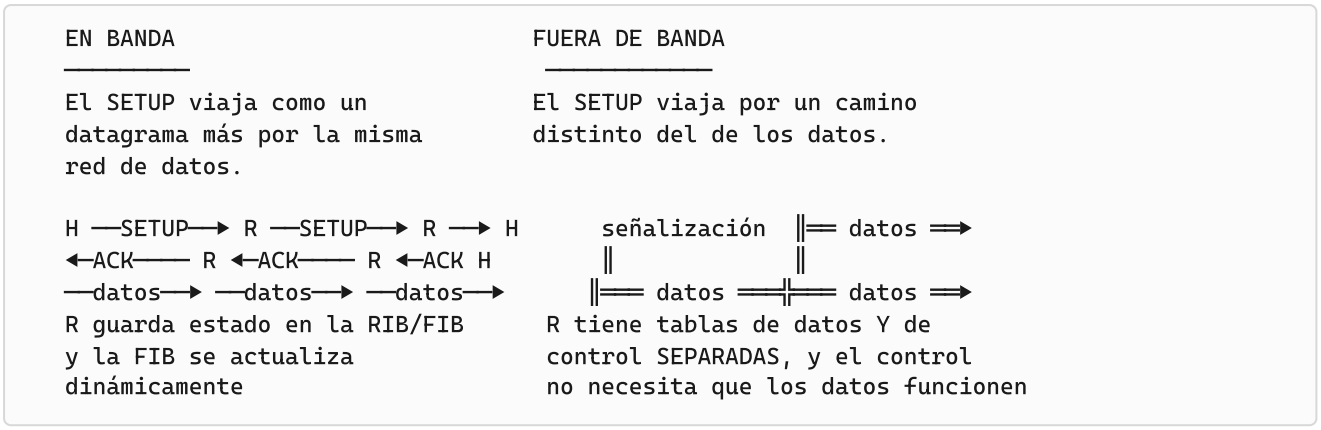
Por qué el VC es más rápido de buscar. Una etiqueta es una **clave primaria**: `tabla[47]`. Una tabla de prefijo es una búsqueda con coincidencia parcial: hay que preguntar a las N entradas "¿cuál de ustedes tiene estos bits como prefijo?" y quedarse con la de prefijo más largo. En hardware, la segunda se resuelve en TCAM (que compara N entradas en paralelo y cuesta caro en silicio); la primera, en una RAM normal o en un simple array indexado. **El VC gana la comparación de tabla y pierde en número de entradas y en flexibilidad.**

Y por qué el VC no escala. Porque su unidad de estado es **la conexión**, no el **prefijo**. Un solo prefijo /24 con 254 hosts es **una** entrada de tabla de datagramas; en el mundo real ese /24 tiene **toda la Internet** detrás. Ese mismo tráfico en VC necesita **una entrada por cada par de extremos que quiera conversar**. Es un ataque a la escalabilidad: el estado crece con el **tráfico y los usuarios**, no con el **espacio de direcciones**. Está en el §2.1 con números.

Por qué esta es la diferencia profunda. El plano de datos de un router moderno tiene una FIB de **millones** de entradas, todas en TCAM. Si el estado estuviera organizado por flujo —que es lo que haría falta para "darle calidad a este TCP"— el router tendría que tener una entrada por cada flujo de Internet, y eso no cabe en silicio. Es exactamente el error de **IntServ** (`RFC 2211`): reservas por flujo que no escalaron. **La razón por la que IP no hizo VC no es que los VC sean malos: es que el estado por flujo no escala.**

1.5 El establecimiento: en banda o fuera de banda

Como en la Clase 24 (§1.5), el setup de un VC puede ir por el mismo canal que los datos o por uno propio.



En banda (ejemplo real): el signaling de Frame Relay SVC y la señalización ATM UNI/PNNI. El mensaje de setup es un paquete que viaja por la red como cualquier otro; los routers lo encaminan, tabla de datos incluida. La consecuencia es la de siempre: **si la red de datos se rompe, el setup también**, y no podés establecer un circuito por un camino que no funciona — lo cual es una paradoja notable: **el circuito que querés crear es precisamente el que arregla el problema**.

Fuera de banda (ejemplo real): la red telefónica clásica. La voz viaja por **circuitos dedicados** conmutados; el control (establecer la llamada) viaja por una **red de señalización separada**, con enlaces y nodos propios. La consecuencia práctica es la que hay que decir siempre: **si se cae la red de señalización, no se puede establecer ninguna llamada aunque todos los circuitos de voz estén libres.** El plano de datos está perfecto y no puede usarse para nada nuevo.

El caso límite: el enlace dedicado. Un enlace punto a punto dedicado (*leased line*) es un circuito con capacidad **1** y sin multiplexado: sólo puede haber una comunicación a la vez, y el camino no se negocia porque **no hay nada que negociar** — la salida es siempre la misma. Es por eso un circuito conmutado en su forma más degenerada, y por eso su setup es **fuera de banda** por definición: lo instala el operador (o el administrador del cable), no un protocolo.

Y el razonamiento es el mismo del `2007-12 P3`. Ese examen pide comparar **conmutación de circuitos, de paquetes y de mensajes**, y el enlace dedicado es el caso extremo de conmutación de circuitos. La misma comparación aplica:

```
CIRCUITOS (incluye el enlace dedicado)
✓ Camino fijo,BW garantizado, retardo acotado, orden garantizado
✓ No hace falta dirección completa en cada paquete
✗ Ancho de banda reservado aunque no se use  →  inefficient con
  tráfico en ráfagas
✗ Establecimiento previo  →  retardo de setup antes del primer dato
✗ Un enlace dedicado = 1 enlace por PAR de nodos = O(n²) enlaces
✗ El equipo de red tiene que asignar y mantener los circuitos

PAQUETES (datagramas)
✓ Sin setup: el primer dato sale ya
✓ El multiplexado estadístico aprovecha los picos de los otros
✓ Un fallo de enlace no rompe la conexión: se busca otro camino
✗ Sin garantías: retardo, jitter, pérdidas y orden no están acotados
✗ Decisión por paquete (más trabajo en el router)
✗ Reordenamiento y cabecera completa por paquete

MENSAJES
✓ Coalescing: uno por mensaje de aplicación, no por paquete
✓ Store-and-forward en cada nodo
✗ Cada nodo debe conocer la ruta completa  →  no escala
✗ Retardo grande: el mensaje entero debe llegar al nodo intermedio
  antes de que pueda reenviarlo
```

1.6 ⚠️ Actualización a 2026

Sigue igual, de memoria:

- **Internet es conmutación de datagramas.** Forwarding por prefijo, sin estado, caminos distintos por paquete. No cambió.
- Las dos familias y sus ventajas/desventajas (reserva de recursos vs. multiplexado estadístico; control de admisión).
- El **VC necesita estado por router** y **establecimiento previo**.
- La etiqueta de VC es **local a cada router**.

Cambió:

- **MPLS sigue desplegándose masivamente, pero en carriers, no en Internet.** `RFC 3031` (arquitectura), `RFC 3032` (pila de etiquetas) y `RFC 4364` (L3VPN) siguen vigentes. Es el VC por excelencia de la infraestructura: el LSP se establece con LDP o RSVP-TE y los datos reenvían **sólo por etiqueta**.
- **El plano de control de MPLS se está moviendo a Segment Routing.** `RFC 8402` (arquitectura, 2018) y `RFC 8986` (SRv6) llevan la idea del VC hasta el extremo: **el estado queda sólo en el borde de ingreso**, y el forwarding de los nodos intermedios es "de SID a SID" sin protocolo de control por salto. **SRv6 codifica los SID dentro de la cabecera IPv6 y elimina el MPLS por completo.** ⇒ Consecuencia conceptual fuerte: **la frontera entre "routing" y "forwarding" se vuelve una convención de diseño, no una necesidad física.**
- **En datacenter, los VC por MPLS fueron reemplazados por EVPN-VXLAN** (`RFC 7432` + `RFC 7348`), que es el mismo juego —etiqueta, estado en el borde, replicación por copia— sobre **UDP** en lugar de MPLS. La etiqueta sigue siendo clave (VNI), pero ya no es MPLS.

- **IntServ es la demostración histórica de que el VC no escala** (`RFC 2211`): mantener estado por flujo en cada router es insostenible, y por eso **DiffServ ganó**. Lo que quedó vivo del VC es la **reserva de recursos en el enlace**, no por flujo: **traffic shaping y policing** (`RFC 2474` / `2475`) marcar el tráfico en el borde y el core aplica una PHB. ⇒ **Sereserve la idea, no el mecanismo por flujo.**
- **El IPv6 Flow Label es el VC que nunca se usó.** La cabecera IPv6 tiene un campo **Flow Label** de 20 bits (`RFC 8200`), que según la especificación debería identificar "flujos que necesitan tratamiento especial". Por defecto es **0** y ningún router de Internet lo usa para reenviar: **el mundo eligió datagramas hasta en el diseño del campo de VC.** Buen ejemplo para el examen de "concepto que existe en el protocolo pero no se usa".
- **El ECMP se comió parte del argumento.** Con ECMP, el datagrama ya reparte un flujo entre varios next hops, así que la "ventaja" del VC de que "el camino es fijo" es **cada vez menos una ventaja**: es una diferencia de *cuánto control* se tiene, no de *si se puede*.

Consecuencia para el examen: la respuesta clásica (datagramas = sin estado ybest-effort; VC = con estado y QoS garantizado) **se sigue preguntando y se sigue respondiendo igual**. Lo que suma es una línea: "*hoy el VC persiste como MPLS en carriers y como overlay (VXLAN/EVPN) en datacenter, pero el estado se concentró en el borde con Segment Routing; y el que no escaló fue el VC con reserva por flujo (IntServ), no el VC en sí.*"

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Entradas de tabla para 1000 hosts: VC vs. datagramas

Datos. Una red interna con **1000 hosts**, todos en la misma LAN lógica (una sola subnet), detrás de un router de borde que los conecta a Internet. Se comparan los dos esquemas.

Pregunta: ¿cuántas entradas de tabla necesita cada router?

Resolución, esquema A — conmutación de datagramas.

```
Los 1000 hosts están en 192.168.10.0/24 (una sola subnet).

FIB del router de borde:
┌──────────────────────┬──────────────────────────┐
│ 192.168.10.0/24      │ eth0 (directamente conn.) │
│ 0.0.0.0/0            │ eth1, next hop 200.1.1.1  │
└──────────────────────┴──────────────────────────┘
                              ▲
                ¡ 2 entradas para 1000 hosts y para
                  todo Internet, gracias a la AGREGACIÓN !

Cada host que habla con Internet usa la entrada 0.0.0.0/0.
Los 1000 hosts comparten la misma entrada porque comparten
prefijo. Nada en la tabla dice "este host".
```

Y si cada host estuviera en su propia subnet (por ejemplo, un /30 por host), entonces sí habría **una entrada por host**: 1000 entradas. Ese es el peor caso del datagrama.

Resolución, esquema B — circuitos virtuales.

Para que **cualquiera de los 1000 hosts pueda hablar con cualquier otro** a través de la red conmutada, hace falta un VC por cada par:

```
número de circuitos entre pares de hosts:

Si el tráfico es UNIDIRECCIONAL y todos hablan con todos:

     N × (N − 1) = 1000 × 999 = 999 000 circuitos

     ┌──────────────────────────────────────┐
     │  Tabla de VC del router:             │
     │  etiqueta 47      → VC 91, eth1      │
     │  etiqueta 91      → VC 22, eth2      │
     │  etiqueta 22      → VC 47, eth3      │
     │  ...                                 │
     │  999 000 entradas                    │
     └──────────────────────────────────────┘

Si el circuito es BIDIRECCIONAL (lo normal):

     N × (N − 1) / 2 = 1000 × 999 / 2 = 499 500 circuitos
```

La comparación, que es el punto de la clase:

	Datagramas	VC
Unidad de estado	el **prefijo**	el **par de extremos**
Entradas para 1000 hosts en una subnet	**2**	hasta **999 000**
Entradas para 1000 hosts en subnets separadas	**1000**	**999 000**
Entradas para **toda Internet**	**millones de prefijos** (agregables)	**una por conexión**
Crecimiento	con el **espacio de direcciones**	con los **usuarios y el tráfico**

Interpretación: el VC necesita **tres órdenes de magnitud** más estado, y su estado **no se puede agregar**: no hay forma de decir "estos 1000 hosts se parecen en algo" porque la etiqueta no tiene estructura. El datagrama sí, porque el prefijo **es** una agrupación. **Ésta es, literalmente, la razón por la que Internet no es una red de circuitos virtuales.** Y es la misma razón por la que el enrutamiento **inter-AS** es path-vector y no distance-vector: en un dominio grande, la métrica por flujo o por vecino se vuelve inmanejable.

2.2 Ejercicio 2 — ¿Qué pasa si se cae un enlace del camino de un VC?

Datos. Un VC establecio H1 ↔ H4 con la ruta H1 – R1 – R2 – R3 – H4. Se cae el enlace R2–R3.

Resolución — paso a paso, comparando los dos esquemas.

Opción A: mantener el VC (como haría Frame Relay con una PVC, o MPLS).

```
t0   el enlace R2–R3 se cae.

t0+  Los paquetes con etiqueta 47 que llegan a R2 seeking salir
     por el enlace caído: se DESCARTAN. El VC sigue "vivo" en
     las tablas: R1, R2, R3 no se enteran de nada.

t0..t+k   Para que alguien lo note hace falta un mecanismo:
     • Frame Relay: mensajes de keepalive cada ~10 s. Si no
       llegan, el circuito se declara DOWN.
     • MPLS con RSVP-TE y LSP de respaldo: Fast Reroute.
       Un camino secundario ya estaba establecido, así que
       la transición es de milisegundos.
     • MPLS con LDP: el LDP se reconecta y vuelve a
       señalizar. Segundos.

t+k   Detección: cada router involved notifica al head-end
     (o al protocolo de señalización).

t2   Si hay respaldo: el VC pasa a usar el camino nuevo, y el
     tráfico se reanuda. Los paquetes perdidos en el hueco
     NO se recuperan: el VC no retransmite.

Si NO hay respaldo: el VC queda DOWN y hay que
re-establecerlo con un nuevo setup. La aplicación tiene que
detectar la caída (timeout) y reconectar.
```

Opción B: red de datagramas.

```
t0   el enlace R2–R3 se cae.

t0+  Los paquetes que iban por ahí se pierden. Los SIGUIENTES
     van por otro camino, si la tabla ya se actualizó. Si la
     tabla todavía no se actualizó, también se pierden
     (la ruta vieja apunta al enlace caído).

t1   El plano de control detecta el fallo y recalcula.
     Con OSPF, los LSA actualizados en ~5 s a 40 s, según el
     temporizador hello/dead. Con BGP, minutos.

t2   La FIB nueva se instala. A partir de ahí, el tráfico
     fluye por el camino nuevo SIN que nadie haya negociado
     nada: no hay conexión que re-establecer.

Lo que se pierde: los paquetes que estaban en vuelo.
Eso lo resuelve TCP con un timeout o 3 dup-ACKs. Nada
necesita renegociar.
```

La comparación, y es la respuesta:

	VC	Datagramas
Detección del fallo	**explícita**: keepalive, señalización o un LSP de respaldo ya armado	**implícita**: los mensajes dejan de llegar, o el siguiente paquete simplemente no sale
Convergencia	hay que **re-establecer**: segundos si hay respaldo, más si no	**automática**: el siguiente paquete ya busca otro camino
Recuperación de datos	**no hay**: el VC no retransmite	la **capa de transporte** (TCP) reenvía
Aplicación	puede tener que **reconectar**	no se entera

Interpretación: el VC es **más eficiente cuando todo anda bien** (búsqueda O(1), camino conocido, sin reordenamiento, calidad garantizada) y **más frágil cuando algo se rompe**, porque el estado hay que reconstruirlo y hay que reconstruirlo *de verdad*. El datagrama es al revés:algo más de trabajo por paquete, pero ningún estado que pueda quedar inconsistente. **Ese es el intercambio clásico entre eficiencia y robustez, y es exactamente por lo que Internet eligió robustez.**

2.3 Ejercicio 3 — Un circuito virtual multipunto

Datos. Un servidor de streaming quiere enviar el mismo flujo a 500 espectadores. ¿Se puede con un solo VC? ¿Qué cambia en los routers?

Resolución. Un VC normal es **punto a punto**: un puerto de entrada, un puerto de salida. Si un VC tiene que llegar a varios destinos, hace falta una de dos formas, y en las dos **el router tiene que dejar de ser un conmutador**:

```
(a) MULTIPOINT-TO-POINT  (bidireccional, "árbol" compartido)
    Todos los que están en el VC pueden hablar entre sí.

          ┌── V2 ──┐
          │        ▼
R1 ──────┼── V1 ──┼── V3 ── R4
          │        ▼
          └── V4 ──┘

    En R1: UNA entrada  VC 100 → "copiá a las interfaces eth0..eth3"
    En los otros: cada uno tiene su VC y reenvía uno a uno.
    El tráfico entre V2 y V3 pasa por R1 y se duplica si
    van en sentidos distintos.

(b) POINT-TO-MULTIPOINT  (unidireccional, "el que manda empuja")
    Un emisor, muchos receptores. El emisor no recibe.

       R1 ── V1 ──┬── V2 ──► V2
                  ├── V3 ──► V3
                  └── V4 ──► V4

    En R1: UNA entrada  VC 100 → "reenviá a eth0, eth1 y eth2"
    Coste: si las ramas son enlaces distintos, el ancho de banda
    se multiplica por 500. Si comparten un enlace, se copia en
    el buffer de salida (y el router tiene que tener RAM para
    500 copias del mismo paquete).
```

Lo que hay que decir:

1. **Un solo VC multipunto reduce muchísimo la tabla**: una entrada en vez de 500. Ese es su appeal.
2. **Pero rompe el modelo de conmutación**: un router que copia a N salidas necesita **memoria proportional a N** (o al menos un buffer de replicación), y necesita poder decidir a quién le copia (si el grupo es dinámico, necesita un estado de membresía, que es exactamente el estado que el VC multipunto se supone que elimina).
3. **Y el ancho de banda se multiplica** si las ramas son independientes.
4. **El equivalente en el mundo de datagramas es el multicast**, y funciona *exactamente por la razón contraria*: se paga estado (quien pertenece al grupo) para **no** pagar ancho de banda, y se resuelve con **flooding + prune** (§1.6 de la Clase 24) y no con un VC. La historia de IP multicast es la historia de intentar hacer multicast **sin** estado por VC.

> En clase, cuando te pidan "un VC multipunto", dibujá las dos formas (árbol bidireccional y empuje unidireccional), y sumá la comparación con multicast. Es un ejercicio que aparece en varios exámenes de ATM y Frame Relay.

2.4 Ejercicio 4 — El enlace dedicado como caso extremo

Enunciado. ¿Es lo mismo un "enlace dedicado" que un circuito conmutado? ¿Y un circuito virtual? Justificá.

Resolución. Los tres son de la familia de **conmutación de circuitos** — camino fijo y dedicada capacidad — pero difieren en cuánto es el circuito y quién lo instala:

```
ENLACE DEDICADO                 CIRCUITO VIRTUAL CONMUTADO
_______________                 __________________________

Un cable físico entre           Un circuito lógico dentro de
dos nodos.                       un enlace compartido.
                                 Switches en el medio eligen
dedicated = no comparte          por dónde sale el circuito.
Capacidad fija para ese par.     Comparte con otros circuitos.

El "setup" es el CARLEO del      El "setup" es un protocolo
cable: lo instala el operador    de señalización entre los
o el administrador.              extremos, en línea.

1 enlace por PAR de nodos.       1 enlace para N nodos
n nodos → n(n-1)/2 enlaces.      (con switches de N puertos).
```

Por qué el enlace dedicado es el mismo razonamiento que el `2007-12 P3`. Porque los tres casos de aquel examen (circuitos, paquetes, mensajes) son la misma comparación en distintos puntos del espectro:

```
               CIRCUITOS      PAQUETES       MENSAJES
               _________      ________       ________

Unidad          sesión         paquete        mensaje de
                                             aplicación
Camino          FIJO           por paquete    completo
Setup           SÍ             NO             NO
BW           reservado       Statistical    reservado
Reordena.       NO             SÍ             NO
Multiplexado    NO             SÍ             según

El enlace dedicado es el extremo izquierdo del todo:
circuito con capacidad 1, sin multiplexado y sin setup en línea.
```

El cálculo que lo mata. Con **n** nodos:

- Enlace dedicado: **n(n−1)/2** enlaces. Para 20 nodos: `20·19/2 = 190` enlaces físicos.
- Conmutación de circuitos: **1** switch, **n** enlaces, y los circuitos se crean y se liberan según hacen falta.
- Conmutación de paquetes: **1** switch, **n** enlaces, y los enlaces se comparten dinámicamente sin reservar nada.

O sea: **190 enlaces contra 20.** El enlace dedicado es la solución correcta cuando el tráfico es constante y de volumen alto (y aun así: 190 leases que pagar). Es la solución **equivocada** para el tráfico en ráfagas de una red de oficinas, porque el 90 % del tiempo estás pagando una capacidad que no usás. **Ese es el mismo argumento del multiplexado estadístico, sólo que con una red de 190 enlaces en vez de uno.**

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-02, Pregunta 4 (9 puntos) — datagramas vs. circuitos virtuales

> **Fuente:** Examen `eirc0902` / `sirc0902` — 2 de febrero de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0902.pdf **Texto del examen:**
>
> > **a)** Describa la conmutación de paquetes basada en: **i.** Datagramas **ii.** Circuitos virtuales. **b)** Explique ventajas y desventajas de cada opción desde el punto de vista de la asignación de recursos en la red.

Solución oficial (b, literal del PDF):

> Básicamente las redes de datagramas son "de mejor esfuerzo", es decir que consumen los recursos disponibles, y en caso de congestión degradan el servicio por igual a todos los usuarios; aprovechan bien el multiplexado estadístico del tráfico en la red. Las redes de VC ofrecen garantía de QoS basadas en la reserva de recursos, y por lo tanto protegen a los usuarios de las situaciones de congestión, a costo de disponer de más recursos o rechazar conexiones (control de admisión).

Sobre la parte a): el PDF de solución **copia el enunciado de la parte a) sin contenido** (remite a las transparencias del curso). Lo que sigue para a) es **solución propia**.

Parte a) — resolución

i. Datagramas. Cada paquete se reenvía por decisión propia, leyendo **su propia dirección destino** y consultando la tabla (en IP, *longest prefix match*). **No hay conexión, no hay estado en el router, no hay negociación previa**: el emisor manda el primer paquete sin handshake. Consecuencias: los paquetes de un mismo flujo **pueden tomar caminos distintos** (y entonces pueden llegar desordenados); si la red cambia, los siguientes paquetes ya aprovechan el cambio sin renegociar nada; y el router no guarda información entre paquetes, así que un reinicio de un router no rompe nada.

ii. Circuitos virtuales. Antes de enviar datos hay un **establecimiento previo**: un intercambio de mensajes de señalización que recorre el camino del destino al origen y vuelve, y que **reserva recursos** en cada enlace del camino. Los datos viajan después identificados por una **etiqueta de VC** (*VC label*), que es **un número corto, no una dirección**, y **todos los paquetes de esa conexión siguen el mismo camino**. Cada router del camino guarda **estado** para ese circuito (etiqueta de entrada → etiqueta y puerto de salida), y ese estado **no existe** en un router de datagramas.

La frase que resume la diferencia, y que hay que decir: en **datagramas la decisión se toma por paquete** (la unidad de estado es el prefijo); en **circuito virtual la decisión se tomó una vez por**

conexión (la unidad de estado es el circuito, identificado por una etiqueta). Todo lo demás —ventajas, desventajas, costo, escalabilidad— se deduce de ahí.

Parte b) — análisis de la solución oficial

La solución oficial es **excelente** y contiene las tres ideas que hay que dar:

1. **Datagramas = mejor esfuerzo**: consumen los recursos disponibles y "en caso de congestión **degradan el servicio por igual a todos los usuarios**". O sea: **ningún usuario tiene prioridad**; el router no sabe a quién pertenece el paquete. Eso es *best-effort* en el sentido fuerte.
2. **Datagramas = bueniplexado estadístico**: aprovechan bien los recursos porque **el tráfico en Internet es en ráfagas** y las capacidades que se reservan dinámicamente se comparten. Si dos flujos usan el mismo enlace en instantes distintos, el ancho de banda está aprovechado.
3. **VC = garantía de QoS por reserva**, que "protege a los usuarios de las situaciones de congestión" pero "**a costo de disponer de más recursos o rechazar conexiones** (**control de admisión**)".

Qué agregaría:

- **El costo en memoria del router.** El VC no sólo reserva ancho de banda: reserva **estado**. Cada router tiene que guardar entradas de tabla por circuito. Ese estado es el que no escala (§2.1: hasta **999 000** circuitos para 1000 hosts), y por eso la reserva de recursos por VC tiene un techo muy bajo en Internet.
- **La asimetría entre "rechazar conexiones" y "degradar a todos".** El control de admisión es una decisión **explícita y temprana**: si no hay recursos, el usuario recibe un "no". El *best-effort* es una decisión **implícita y tardía**: el usuario no se entera hasta que ve que le faltó. Para el proveedor, el control de admisión es **predecible**; para el usuario, es **injusto**.
- **El coste de la conmutación de circuitos con recursos compartidos es el número de circuits que deben estar "listos"**. Si un VC necesita 2 Mbit/s garantizados y hay 100 usuarios, hacen falta 200 Mbit/s **reservados** en el enlace, o de lo contrario hay que **rechazar** el VC número 101. Ese es exactamente el "costo de disponer de más recursos" de la solución oficial, y es la razón por la que un operador con una red de 1 Gbit/s no puede ofrecer QoS a 100 usuarios de 20 Mbit/s aunque el promedio de uso sea 1 Mbit/s.
- **El caso en el que el VC gana de verdad: el tráfico en ráfagas con requirement de calidad.** Si 90 % de la capacidad reservada está ociosa, el VC está desperdiciando; pero si hay una aplicación que necesita un piso de calidad (voz en tiempo real, un enlace de respaldo), la reserva es la única manera de **garantizarlo**. El problema nunca es la idea del VC: es hacerlo **con reserva por flujo** y no por clase.

> **Tope para 9 puntos** (a ~4, b ~5): en (a), las dos definiciones con sus tres condiciones (decisión por paquete / etiqueta + estado + setup). En (b), *best-effort* y multiplexado estadístico para datagramas; reserva de recursos, garantía de QoS y **control de admisión** para VC; y el costo (más recursos o rechazo) que es la palabra clave. Con sólo decir "datagramas es mejor esfuerzo y VC garantiza calidad" son 4-5 de 9.

3.2 Examen 2007-12, Pregunta 3 (10 puntos) — circuitos, paquetes y mensajes

> **Fuente:** `eirc0712` / `sirc0712` — 27 de diciembre de 2007. **Texto:**
>
> > Explique en que consiste la conmutación de circuitos, conmutación de paquetes y conmutación de mensajes. Realice un análisis comparativo.

Nota: el PDF de solución no desarrolla esta pregunta. El desarrollo completo de la conmutación de circuitos / paquetes / mensajes es la **Clase 4**; acá va el resumen en la forma de tabla de §1.5, más el enlace al razonamiento del enlace dedicado (§2.4).

Lo que hay que decir, en una frase cada una.

- **Conmutación de circuitos:** se establece un camino dedicado de principio a fin antes de transmitir, y todo el tráfico de esa sesión va por ese camino, con recursos reservados. La red telephone clásica es el ejemplo.
- **Conmutación de paquetes:** no hay camino previo; cada paquete lleva la dirección destino y el switch decide por dónde sale. Es la conmutación de datagramas si las decisiones son independientes, y la de **VC** si hay un circuito establecido con etiqueta. Internet es de paquetes.
- **Conmutación de mensajes:** la unidad conmutada es el **mensaje completo de la aplicación**. El nodo intermedio tiene que **recibir el mensaje entero** antes de poder reenviarlo (*store-and-forward*), y necesita **conocer la ruta completa** hasta el destino final, porque cada mensaje lleva un nombre de destino y una lista deilhitos. **No escala** y agrega retardo, a cambio de una muy buena eficiencia en el número de paquetes por mensaje (**coalescing**).

La tabla comparativa es la de §1.5. Y si te piden un ejemplo de cada una: **circuitos** = la llamada telefónica (o un enlace dedicado); **paquetes** = un datagrama IP, o un VC de MPLS; **mensajes** = un email que viaja como un único bloque por la red (las redes *store-and-forward* de los origen de los años '80).

3.3 Examen 2010-02, Pregunta 2c — las dos formas de servicio de capa de red

> **Fuente:** `eirc1002` / `sirc1002` — 2 de febrero de 2010. **Texto:**
>
> > c) Describa brevemente las dos formas de prestar servicios de capa de red.

Es exactamente el tema de esta clase. Las dos formas son:

1. **Servicio de conmutación de datagramas** (*datagram service*): sin conexión y sin estado, cada paquete se reenvía por decisión propia con longest prefix match, los paquetes de un mismo flujo pueden tomar

caminos distintos, no hay reserva de recursos y el servicio es *best-effort*. **Es el servicio de Internet.**

2. **Servicio de conmutación de circuitos virtuales** (*virtual circuit service*): con conexión, hay establecimiento previo, se establece un camino lógico que todos los paquetes siguen identificados por una etiqueta, cada router guarda estado para ese circuito, y puede haber reserva de recursos con control de admisión y garantías de calidad. **Es el servicio de Frame Relay, ATM y MPLS.**

Nota: el enunciado de 2010-02 es ambigüo y muchos alumnos lo responden como "conexión / sin conexión". La lectura correcta es "datagramas / circuitos virtuales", porque son las dos formas de conmutación dentro de una **red de paquetes** — es la misma pregunta que la `2009-02 P4` de esta clase, con otra redacción.

PARTE 4 — CIERRE

📌 Resumen

1. Las dos familias se distinguen por **una pregunta**: ¿la decisión depende del **paquete** o de un **camino previamente establecido**?
2. **Datagramas**: sin conexión, sin estado, decisión **por paquete**, caminos distintos para el mismo flujo, tabla por **prefijo** y **agregable**. Es Internet entero.
3. **Circuitos virtuales**: **establecimiento previo** + **etiqueta** + **estado en cada router**. La etiqueta es un número **local** a cada router, no una dirección.
4. La tabla por VC es una **lectura O(1)**; la de prefijo, un **longest prefix match** en TCAM. El VC gana la búsqueda y pierde en **cantidad de entradas** (999 000 circuitos para 1000 hosts) y en **agregabilidad**.
5. La **razón profunda** de que Internet no sea de VC: el estado por flujo **no escala en silicio**. Es el error de IntServ (`RFC 2211`).
6. El **enlace dedicado** es el circuito conmutado en su forma más degenerada: capacidad 1, sin multiplexado, `n(n−1)/2` enlaces para `n` nodos, y setup **fuera de banda**.
7. El VC es **más eficiente si nada se rompe y más frágil si algo se rompe**: su estado hay que reconstruirlo; el datagrama no tiene estado que pueda quedar inconsistente.
8. **Multipunto** rompe el modelo de conmutación (hay que copiar a N salidas, con memoria proporcional a N); su equivalente escalable en el mundo datagrama es el **multicast** con *flood-and-prune*.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Qué distingue formalmente a un router de datagramas de uno de circuitos virtuales?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Por qué la etiqueta de VC es "local a cada router" y qué consecuencia tiene?

<details>
<summary>Respuesta</summary>
</details>

3. 1000 hosts en una misma subnet. ¿Cuántas entradas de tabla necesita cada esquema?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué Internet no es una red de circuitos virtuales?

<details>
<summary>Respuesta</summary>
</details>

5. ¿Qué le pasa a un VC si se cae un enlace de su camino? ¿Y a un flujo de datagramas?

<details>
<summary>Respuesta</summary>
</details>

6. ¿Qué es un circuito virtual multipunto y por qué rompe el modelo de conmutación?

▶ Respuesta

7. ¿El enlace dedicado es lo mismo que un circuito virtual? Explicá con un cálculo.

▶ Respuesta

8. En 2026, ¿sigue siendo verdad que "Internet es conmutación de datagramas"? ¿Qué cambió en el mundo de los VC?

▶ Respuesta

🔗 Referencias

- **Examen `eirc0902` / `sirc0902`** — P4 (9 pts): conmutación por datagramas vs. circuitos virtuales y ventajas/desventajas desde la asignación de recursos. La pregunta central de esta clase.
- **Examen `eirc0712` / `sirc0712`** — P3 (10 pts): conmutación de circuitos, de paquetes y de mensajes, con análisis comparativo (desarrollada en la Clase 4; el enlace dedicado es el caso extremo de circuitos).
- **Examen `eirc1002` / `sirc1002`** — P2c: las dos formas de prestar servicios de capa de red. · **Examen `eirc1103` / `sirc1103`** — P1: las dos funciones clave de la capa de red.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red" — §5.1 las dos familias de redes: Circuitos virtuales y conmutación de datagramas.
- **RFC 791** — Internet Protocol (1981): el modelo de datagrama, sin conexión y best-effort.
- **RFC 3031** — MPLS Architecture (2001) · **RFC 3032** — MPLS Label Stack (2001) · **RFC 4364** — MPLS L3VPN.
- **RFC 2211** — IntServ (1997): el modelo de reserva por flujo que **no escaló**, y la razón por la que DiffServ ganó.
- **RFC 2474 / RFC 2475** — DS field y arquitectura de DiffServ: lo que quedó de la QoS, pero por **clase**, no por flujo.
- **RFC 8402** — Segment Routing Architecture (2018) · **RFC 8986** — SRv6 Network Programming (2022): estado sólo en el borde de ingreso.
- **RFC 8200** — IPv6 (2017): el campo **Flow Label**, el VC que nunca se usó.
- **RFC 7432 / RFC 7348** — EVPN / VXLAN: el VC del datacenter sobre UDP, que reemplazó a MPLS ahí.
- **Clase siguiente:** 26 — IPv4: el datagrama y las direcciones